Agentic Derivatives Platform - Data Pipeline & Agent Architecture

MVP Implementation Specifications

OVERVIEW

This document provides detailed specifications for the MVP data pipeline and AI agent architecture. The system focuses on building a solid foundation for market data ingestion, analysis, and signal generation without implementing specific trading strategies. The architecture uses managed services (Supabase, Railway, Redis) for rapid development while maintaining scalability.

---

1. TECHNOLOGY STACK

Core Platform Architecture

The platform uses a managed services approach to minimize operational complexity while providing enterprise-grade capabilities for market data processing and AI agent coordination.

```
Infrastructure Components:
├── Supabase: Database + Auth + Real-time subscriptions
├── Railway: Backend services (FastAPI microservices) + Redis hosting
├── Vercel: Frontend hosting and deployment
├── Redis: Pub/sub messaging + hot data caching (managed via Railway)
└── Managed approach: Focus on features over infrastructure management

Technology Stack:
├── Backend: Python + FastAPI + LangChain + Pydantic
├── Frontend: React + TypeScript + Vercel
├── Database: PostgreSQL (via Supabase) + Redis
├── AI/ML: LiteLLM for multi-provider routing + LangChain agents
└── APIs: Alpaca (market data) + Brave Search (news/sentiment) + FRED (economic data)
```

---

2. SYMBOL UNIVERSE

Market Coverage Strategy

The MVP tracks 21 carefully selected symbols that provide comprehensive market intelligence across major themes: broad market exposure, sector rotation, volatility regime, and leveraged/inverse positioning.

```
Symbol Categories:

Core Market Indices (3):
├── SPY: S&P 500 broad market proxy
├── QQQ: Nasdaq 100 technology focus
└── IWM: Russell 2000 small cap exposure

Sector ETFs - XL Series (9):
├── XLK: Technology sector
├── XLF: Financial sector
├── XLE: Energy sector
├── XLV: Healthcare sector
├── XLI: Industrial sector
├── XLY: Consumer Discretionary
├── XLP: Consumer Staples
├── XLU: Utilities sector
└── XLRE: Real Estate sector

Volatility Products (4):
├── VIX: CBOE Volatility Index
├── UVXY: 1.5x leveraged short-term VIX futures
├── VXX: 1x short-term VIX futures
└── SVXY: -0.5x inverse short-term VIX futures

Leveraged/Inverse ETFs (5):
├── TQQQ: 3x leveraged Nasdaq
├── SQQQ: 3x inverse Nasdaq
├── SH: 1x inverse S&P 500
├── SPXL: 3x leveraged S&P 500
└── SPXU: 3x inverse S&P 500

Total: 21 symbols providing comprehensive market intelligence
```

---

3. DATA INGESTION LAYER

Multi-Source Data Collection Framework

The ingestion layer handles real-time data collection from multiple external APIs, managing connections, rate limits, and initial data validation before passing to the normalization framework.

```
Data Ingestion Architecture:

Primary Data Sources:
├── Alpaca Markets API:
│   ├── WebSocket Streams: Real-time quotes, bars, trades for all 21 symbols
│   ├── REST API: Historical data backfill and market status
│   ├── Connection Management: Auto-reconnect with exponential backoff
│   ├── Rate Limits: 200 requests/minute market data, handle gracefully
│   └── Data Types: Live quotes, 1-minute bars, trade data, market status
├── Brave Search API:
│   ├── REST Endpoints: News search and content retrieval
│   ├── Search Strategy: Time-based searches every 2 hours during market hours
│   ├── Query Patterns: Symbol-specific + general market terms
│   ├── Rate Limits: 2000 queries/month free tier, intelligent throttling
│   └── Data Types: News articles, headlines, publication dates, source URLs
└── FRED Economic Data API:
    ├── REST Endpoints: Economic indicators and series data
    ├── Update Schedule: Daily polling for new releases
    ├── Key Indicators: Fed Funds Rate, 10Y Treasury, CPI, unemployment
    ├── Rate Limits: Generous limits, daily polling sufficient
    └── Data Types: Economic series, release dates, historical values

Connection Management:
├── Alpaca WebSocket:
│   ├── Persistent Connection: Maintain 24/7 connection during market hours
│   ├── Subscription Management: Dynamic symbol list management
│   ├── Heartbeat Monitoring: Detect connection drops within 30 seconds
│   ├── Auto-Reconnect: Exponential backoff (1s, 2s, 4s, 8s, max 60s)
│   ├── Error Handling: Log connection issues, attempt graceful recovery
│   └── Data Validation: Basic sanity checks before forwarding to normalization
├── REST API Polling:
│   ├── Brave Search: Scheduled searches with intelligent retry logic
│   ├── FRED Data: Daily economic indicator updates
│   ├── Rate Limit Compliance: Track usage across all endpoints
│   ├── Timeout Handling: 30-second timeouts with retry for critical data
│   └── Circuit Breaker: Temporary suspension after repeated failures

Data Collection Workflows:
├── Market Hours (9:30 AM - 4:00 PM ET):
│   ├── Alpaca WebSocket: Continuous real-time data streaming
│   ├── Brave Search: Every 2 hours for news/sentiment updates
│   ├── Health Monitoring: Connection status checks every 60 seconds
│   └── Data Quality: Real-time validation and gap detection
├── After Hours (4:00 PM - 9:30 AM ET):
│   ├── Alpaca: Reduced frequency polling for overnight moves
│   ├── FRED: Daily economic data updates (typically 8:30 AM ET)
│   ├── Brave Search: Overnight news accumulation
│   └── Maintenance: Connection cleanup and preparation for market open
└── Weekends/Holidays:
    ├── Minimal Data Collection: Economic releases only
    ├── System Maintenance: Data cleanup and optimization
    ├── Historical Backfill: Fill any gaps from market hours
```

```
  └── Preparation: Setup for next trading session

Error Handling & Recovery:
├── Connection Failures:
│   ├── Immediate Retry: Quick reconnection attempt (1-3 seconds)
│   ├── Progressive Backoff: Increasing delays for persistent failures
│   ├── Fallback Strategies: Switch to backup endpoints when available
│   ├── Alert Generation: Notify operations team of extended outages
│   └── Data Gap Tracking: Log missing data periods for later backfill
├── Rate Limit Management:
│   ├── Usage Monitoring: Track API calls against known limits
│   ├── Intelligent Throttling: Slow down requests approaching limits
│   ├── Priority Queuing: Critical data requests take precedence
│   ├── Graceful Degradation: Reduce polling frequency when limited
│   └── Quota Reset Handling: Resume normal operations when limits reset
└── Data Validation:
    ├── Range Checking: Validate prices within reasonable bounds
    ├── Timestamp Validation: Ensure data freshness and proper sequencing
    ├── Duplicate Detection: Identify and filter redundant data points
    ├── Missing Field Handling: Flag incomplete data for quality scoring
    └── Format Validation: Ensure data matches expected schema

Service Architecture:
├── Alpaca Data Service (FastAPI):
│   ├── WebSocket Handler: Manage persistent connections and subscriptions
│   ├── REST Client: Historical data and market status queries
│   ├── Data Router: Forward validated data to normalization framework
│   ├── Health Monitor: Connection status and performance metrics
│   └── Configuration: Dynamic symbol management and polling intervals
├── News Intelligence Service (FastAPI):
│   ├── Brave Search Client: News query execution and result processing
│   ├── Content Parser: Extract relevant information from search results
│   ├── Sentiment Preprocessor: Basic sentiment scoring before AI analysis
│   ├── Rate Limiter: Intelligent API usage management
│   └── Cache Manager: Store recent searches to avoid duplicate queries
└── Economic Data Service (FastAPI):
    ├── FRED Client: Economic indicator polling and historical data
    ├── Release Calendar: Track upcoming economic data releases
    ├── Data Transformer: Convert FRED format to platform standard
    ├── Update Scheduler: Daily polling with smart scheduling
    └── Historical Manager: Maintain economic time series data

Performance Monitoring:
├── Data Flow Metrics:
│   ├── Ingestion Rate: Messages per second by source and symbol
│   ├── Latency Tracking: Time from source to normalization framework
│   ├── Error Rates: Failed requests, connection drops, validation failures
│   ├── Coverage Metrics: Percentage of expected data points received
│   └── Quality Scores: Initial data quality assessment before normalization
├── Connection Health:
│   ├── Uptime Tracking: Connection availability by source
│   ├── Reconnection Frequency: Pattern analysis for connection stability
│   ├── Response Times: API response latency monitoring
```

```
│   ├── Throughput Analysis: Data volume trends and capacity planning
│   └── Alert Thresholds: Automated notifications for degraded performance

Integration with Normalization:
├── Data Handoff: Seamless transfer to Data Quality & Normalization Framework
├── Source Tagging: All data tagged with source, timestamp, and quality indicators
├── Format Consistency: Ensure all sources provide data in expected structure
├── Metadata Preservation: Maintain collection context for quality assessment
└── Performance Feedback: Normalization results inform collection optimization
```

---

4. DATA QUALITY & NORMALIZATION FRAMEWORK

Unified Data Processing Pipeline

This framework handles all aspects of data ingestion, from raw API responses to clean, validated data ready for AI analysis. It provides a single abstraction layer that normalizes data from multiple sources into consistent formats.

Data Quality Framework Components:

```
Temporal Synchronization:
├── Standardize timestamps to UTC ISO 8601 format
├── Handle timezone differences across data sources
├── Align data points to common time intervals
├── Detect and flag timestamp anomalies
└── Maintain microsecond precision for latency tracking

Quality Scoring System:
├── Completeness: Check for missing required fields (0-1 score)
├── Timeliness: Measure data freshness vs expected intervals (0-1 score)
├── Accuracy: Validate against reasonable ranges and patterns (0-1 score)
├── Consistency: Compare across sources for same data point (0-1 score)
└── Overall Quality Score: Weighted average of individual components

Data Standardization:
├── Symbol Normalization: Convert all symbols to standard format
├── Price Formatting: Consistent decimal precision and rounding
├── Volume Standardization: Handle different volume units across sources
├── Metadata Enrichment: Add source, latency, and quality information
└── Error Classification: Categorize and tag data quality issues

Source Management:
├── Alpaca WebSocket: Real-time quotes, bars, trades with auto-reconnect
├── Brave Search API: News and sentiment with rate limiting
├── FRED API: Economic indicators with caching
├── Rate Limiting: Intelligent throttling per source API limits
└── Health Monitoring: Connection status and data flow validation

Missing Data Handling:
├── Gap Detection: Identify missing time intervals or symbols
├── Forward Fill: Use last known value for brief gaps (<5 minutes)
├── Interpolation: Linear interpolation for longer gaps where appropriate
├── Quality Flagging: Mark interpolated data with reduced quality scores
└── Alert Generation: Notify when significant data gaps occur

Standard Data Format:
{
  "symbol": "SPY",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "data_type": "quote|bar|news|economic",
  "source": "alpaca|brave|fred",
  "quality_score": 0.95,
  "latency_ms": 45,
  "data": {
    // Source-specific normalized data
  },
  "metadata": {
    "original_timestamp": "...",
    "processing_time": "...",
    "flags": ["interpolated", "delayed", etc.]
```

```
  }
}
```

---

5. STORAGE & RETRIEVAL ARCHITECTURE

Intelligent Data Lifecycle Management

The storage system uses a two-tier approach optimized for both real-time access and historical analysis. Data flows automatically between hot storage (Redis) for immediate access and warm storage (Supabase) for persistence and complex queries.

```
Storage Architecture:

Data Tier Strategy:
├── Hot Tier (Redis): Real-time data with TTL-based expiration
│   ├── Live market quotes: 5-minute TTL
│   ├── Recent technical indicators: 1-hour TTL
│   ├── Active alerts and signals: 30-minute TTL
│   ├── User session data: 24-hour TTL
│   └── Pub/sub message queues: Immediate distribution
│
└── Warm Tier (Supabase): Persistent storage with intelligent retention
    ├── Historical market data: TTL based on timeframe
    ├── AI-generated insights: Permanent with metadata
    ├── User configurations: Permanent
    ├── Performance tracking: Permanent for analysis
    └── Audit logs: Configurable retention periods

Redis Configuration:
├── Connection: Railway-managed Redis instance
├── Memory Management: LRU eviction policy for cache optimization
├── Persistence: RDB snapshots for critical data recovery
├── Pub/Sub Channels: Topic-based routing for efficient distribution
└── Key Naming Convention: {source}:{symbol}:{data_type}:{timestamp}

Supabase Schema Design:
├── normalized_data: All processed market data with quality scores
├── ai_insights: Agent-generated analysis and predictions
├── alert_history: Historical alerts and user notifications
├── data_quality_metrics: Source reliability and performance tracking
└── user_subscriptions: Real-time data subscription management

Retrieval Patterns:
├── Real-time Access: Redis first, fallback to Supabase
├── Historical Queries: Supabase with Redis caching for frequent requests
├── Bulk Analysis: Direct Supabase queries with pagination
├── Agent Data Access: Subscription-based Redis channels
└── Frontend Updates: Supabase real-time subscriptions

Data Expiration Policies:
├── 1-minute market data: 3-day retention in Supabase
├── 5-minute aggregated data: 30-day retention
├── Hourly data: 1-year retention
├── Daily data: Permanent retention
├── AI insights: Permanent with performance tracking
└── Redis TTL: Automatic cleanup based on data type and usage patterns
```

---

6. STORAGE COORDINATOR SPECIFICATIONS

Central Data Distribution Hub

The Storage Coordinator acts as the central nervous system for all data flow between agents, storage systems, and external consumers. It handles intelligent routing, caching, and real-time distribution while maintaining data consistency across the platform.

```
Storage Coordinator Architecture:

Core Responsibilities:
├── Insight Ingestion: Receive analysis from all AI agents
├── Storage Routing: Determine optimal storage location per data type
├── Real-time Distribution: Publish to Redis channels with smart routing
├── Data Indexing: Maintain searchable catalog of all insights
├── Lifecycle Management: Handle TTL policies and data cleanup
├── Performance Monitoring: Track storage and retrieval metrics
└── API Gateway: Unified interface for data access requests

Redis Channel Strategy:
├── Channel Naming: {category}:{agent_type}:{symbol}
│   ├── insights:vix_analysis:VIX
│   ├── insights:sector_rotation:XLK
│   ├── alerts:high_priority:market_regime
│   └── requests:latest_analysis:SPY
├── Broadcast Channels:
│   ├── insights:all - All agent outputs
│   ├── alerts:critical - Emergency signals only
│   └── market:live - Real-time market data stream
└── Subscription Management: Dynamic channel subscriptions based on user preferences

Supabase Integration:
├── Table Structure:
│   ├── agent_insights: id, agent_type, symbol, insight_data (JSONB), confidence_score, timestamp, expires_at
│   ├── insight_subscriptions: subscriber_id, channel_pattern, filters (JSONB), created_at
│   ├── insight_performance: insight_id, prediction_accuracy, outcome_timestamp, notes
│   └── data_catalog: data_type, source, retention_policy, access_pattern, last_updated
├── Real-time Subscriptions: Automatic frontend updates via Supabase channels
├── Query Optimization: Indexes on timestamp, symbol, agent_type for fast retrieval
└── Data Retention: Automated cleanup based on configurable policies

API Interface Requirements:
├── store_insight(insight_object) -> confirmation_id
│   ├── Validate insight format and required fields
│   ├── Determine storage location (Redis hot vs Supabase persistent)
│   ├── Publish to appropriate Redis channels
│   ├── Update data catalog and indexing
│   └── Return confirmation with storage location details
├── get_latest(agent_type, symbol, filters) -> insight_object
│   ├── Check Redis cache first for immediate response
│   ├── Fallback to Supabase for historical data
│   ├── Apply filters and data quality thresholds
│   └── Cache frequently requested data back to Redis
├── subscribe(channel_pattern, callback_url, filters) -> subscription_id
│   ├── Register real-time subscription with pattern matching
│   ├── Apply data quality and confidence filters
│   ├── Handle callback delivery and retry logic
│   └── Manage subscription lifecycle and cleanup
├── query_historical(agent_types, symbols, time_range, aggregation) -> dataset
│   ├── Execute optimized Supabase queries with pagination
```

```
│   ├── Apply data quality filters and confidence thresholds
│   ├── Handle large result sets with streaming responses
│   └── Cache aggregated results for similar future queries
└── publish_alert(priority, agent_type, message, affected_symbols) -> alert_id
    ├── Validate alert priority and routing requirements
    ├── Publish to appropriate alert channels with fanout
    ├── Store in alert history with metadata
    ├── Trigger user notifications based on subscription preferences
    └── Track alert delivery and user engagement

Standardized Data Format:
{
  "insight_id": "uuid4",
  "agent_type": "vix_analysis|sector_rotation|market_regime",
  "symbol": "VIX|SPY|XLK|...",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "confidence_score": 0.85,
  "data_quality": 0.92,
  "expires_at": "2024-01-15T14:30:00.000Z",
  "insights": {
    // Agent-specific analysis data
    "analysis_type": "volatility_environment",
    "current_value": 18.5,
    "trend_direction": "rising",
    "predictions": {...},
    "supporting_evidence": [...]
  },
  "alerts": [
    {
      "priority": "high|medium|low",
      "message": "VIX approaching 20 threshold",
      "trigger_conditions": {...}
    }
  ],
  "metadata": {
    "processing_time_ms": 45,
    "data_sources": ["alpaca", "fred"],
    "model_version": "1.0.2",
    "correlation_ids": ["related_insight_id_1", "..."]
  }
}

Performance Requirements:
├── Insight Storage Latency: <50ms to Redis, <200ms to Supabase
├── Real-time Distribution: <10ms to publish to Redis channels
├── Query Response Time: <100ms for cached data, <500ms for database queries
├── Throughput Capacity: Handle 100+ insights per second sustained
├── Channel Scalability: Support 1000+ concurrent subscriptions
├── Data Consistency: Ensure eventual consistency between Redis and Supabase
└── Error Recovery: Automatic retry logic with exponential backoff

Error Handling & Monitoring:
├── Storage Failures: Automatic failover between Redis and Supabase
```

```
├── Channel Delivery: Retry logic for failed pub/sub deliveries
├── Data Validation: Reject malformed insights with detailed error logging
├── Performance Monitoring: Track latency, throughput, and error rates
├── Alerting: Notify on storage capacity, performance degradation, or failures
└── Health Checks: Regular validation of storage connections and data integrity
```

---

7. AI AGENT SPECIFICATIONS

Agent Architecture Philosophy

The MVP implements simple, focused agents that analyze specific market conditions and generate insights. Each agent specializes in one area and communicates through the Storage Coordinator rather than direct inter-agent communication.

VIX Ecosystem Analysis Agent

Multi-Dimensional Volatility Intelligence

The VIX Analysis Agent provides comprehensive volatility environment assessment by analyzing the VIX index alongside related volatility products. It generates predictive signals and regime classifications that inform other agents about market stress levels and volatility trading opportunities.

```
VIX Analysis Agent Specifications:

Data Requirements:
├── Real-time Price Data:
│   ├── VIX: Current level, intraday range, volume patterns
│   ├── UVXY: 1.5x leveraged VIX futures performance
│   ├── VXX: 1x VIX futures benchmark tracking
│   └── SVXY: Inverse VIX futures performance indicator
├── Historical Context:
│   ├── 20-day VIX rolling statistics (mean, std dev, percentiles)
│   ├── 252-day lookback for long-term regime classification
│   ├── VIX product correlation analysis and divergence detection
│   └── Volume patterns across volatility product complex
└── External Indicators:
    ├── SPY/QQQ price action for volatility-equity relationship validation
    ├── Sector performance (XL* ETFs) for risk-on/risk-off confirmation
    └── Economic calendar events for volatility catalyst identification

Analysis Framework:
├── Absolute Level Classification:
│   ├── Extremely Low VIX (<12): Complacency warning, vol selling opportunity
│   ├── Low VIX (12-15): Below-average volatility, premium selling favored
│   ├── Normal VIX (15-20): Average environment, balanced approach
│   ├── Elevated VIX (20-25): Above-average volatility, mixed strategies
│   ├── High VIX (25-30): Market stress, volatility buying favored
│   └── Extreme VIX (>30): Crisis mode, defensive positioning only
├── Percentile Analysis:
│   ├── Current VIX vs 20-day percentile rank
│   ├── Current VIX vs 252-day percentile rank
│   ├── Trend analysis: Rising/falling percentile over time
│   └── Mean reversion probability based on extreme percentile readings
├── Trend & Momentum Analysis:
│   ├── VIX rate of change: 1-day, 3-day, 5-day momentum
│   ├── Trend classification: Accelerating, steady, decelerating
│   ├── Reversal probability: Statistical likelihood of trend change
│   └── Volatility clustering: Persistence vs mean reversion regime
└── Cross-Product Analysis:
    ├── VIX vs UVXY/VXX divergence (contango/backwardation signals)
    ├── SVXY performance vs VIX inverse correlation health
    ├── Volume analysis across volatility products for flow intelligence
    └── Premium/discount analysis for volatility product fair value

Predictive Modeling:
├── Short-term Forecasts (1-5 days):
│   ├── VIX range prediction with confidence intervals
│   ├── Mean reversion probability from current levels
│   ├── Volatility expansion/contraction likelihood
│   └── Support/resistance levels for VIX technical analysis
├── Medium-term Analysis (1-4 weeks):
│   ├── Volatility regime sustainability assessment
│   ├── Seasonal volatility pattern integration
│   ├── Event-driven volatility catalyst timing
```

```
│   └── Cross-asset volatility spillover analysis
└── Risk Scenario Modeling:
    ├── Tail risk probability (VIX >30, >40 scenarios)
    ├── Volatility shock absorption capacity
    ├── Market stress test implications
    └── Portfolio protection requirement assessment
```

Output Format:


```
{
  "agent_type": "vix_analysis",
  "symbol": "VIX",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "confidence_score": 0.87,
  "insights": {
    "current_analysis": {
      "vix_level": 18.5,
      "classification": "normal",
      "percentile_rank_20d": 0.65,
      "percentile_rank_252d": 0.42,
      "trend_direction": "rising",
      "momentum_score": 0.73
    },
    "cross_product_analysis": {
      "uvxy_divergence": 0.15,
      "vxx_correlation": 0.94,
      "svxy_performance": -0.18,
      "volume_surge_detected": false,
      "smart_money_flow": "neutral"
    },
    "predictions": {
      "1day_range": {"low": 17.2, "high": 19.8, "confidence": 0.68},
      "1week_regime": {"classification": "normal_to_elevated", "confidence": 0.55},
      "mean_reversion_prob": 0.72,
      "expansion_risk": 0.28
    },
    "regime_assessment": {
      "current_regime": "normal_volatility",
      "regime_stability": 0.75,
      "transition_probability": {
        "to_low": 0.15,
        "to_elevated": 0.35,
        "to_high": 0.10
      }
    },
    "trading_implications": {
      "environment_score": "neutral_to_bearish_vol",
      "volatility_selling_opportunity": 0.45,
      "volatility_buying_opportunity": 0.65,
      "risk_management_urgency": "moderate"
    }
  },
  "alerts": [
    {
```

```
      "priority": "medium",
      "message": "VIX approaching 20 resistance level",
      "trigger_threshold": 19.5,
      "confidence": 0.78
    }
  ]
}
```


Sector Rotation Analysis Agent

Cross-Sector Performance Intelligence

Analyzes relative performance across all XL sector ETFs to identify rotation patterns, leadership changes, and risk-on/risk-off sentiment shifts.

Sector Rotation Agent Specifications:

```
Data Requirements:
├── Sector ETF Performance:
│   ├── All XL* ETFs: 1-day, 5-day, 20-day, 60-day returns
│   ├── Relative performance vs SPY benchmark
│   ├── Volume patterns and institutional flow indicators
│   └── Intraday momentum and strength measures
├── Market Context:
│   ├── SPY/QQQ/IWM performance for market regime context
│   ├── Treasury yields (10Y) for interest rate sensitivity
│   ├── Dollar strength for international exposure impact
│   └── VIX levels for risk appetite assessment

Analysis Framework:
├── Relative Strength Analysis:
│   ├── Sector vs SPY performance ratios across multiple timeframes
│   ├── Momentum persistence: Consistency of outperformance
│   ├── Strength rankings: Ordered sector performance lists
│   └── Divergence detection: Sectors breaking from trends
├── Rotation Pattern Recognition:
│   ├── Growth vs Value: XLK/XLY vs XLF/XLE/XLU rotation
│   ├── Cyclical vs Defensive: XLI/XLF vs XLP/XLU positioning
│   ├── Risk-On vs Risk-Off: Leverage-sensitive vs safe haven flows
│   └── Interest Rate Sensitivity: Duration exposure analysis
├── Leadership Change Detection:
│   ├── New sector leadership emergence (momentum breakouts)
│   ├── Exhaustion signals in current leaders (momentum stalling)
│   ├── Laggard resurrection patterns (oversold sector recovery)
│   └── Broad market vs sector performance divergences
└── Flow Analysis:
    ├── Volume surge detection across sectors
    ├── Smart money vs retail flow differentiation
    ├── ETF creation/redemption activity patterns
    └── Options flow and hedging activity implications

Sector Classification:
├── Growth Sectors: XLK (Technology), XLY (Consumer Discretionary)
├── Value Sectors: XLF (Financials), XLE (Energy), XLI (Industrials)
├── Defensive Sectors: XLP (Staples), XLU (Utilities), XLV (Healthcare)
├── Interest Sensitive: XLF (Financials), XLRE (Real Estate)
└── Cyclical Sectors: XLI (Industrials), XLB (Materials), XLE (Energy)
```

Output Format:

```
{
  "agent_type": "sector_rotation",
  "symbol": "XL_SECTORS",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "confidence_score": 0.82,
  "insights": {
    "sector_rankings": {
      "1day": [
```

```
      {"sector": "XLK", "return": 0.024, "vs_spy": 0.008, "rank": 1},
      {"sector": "XLY", "return": 0.018, "vs_spy": 0.002, "rank": 2},
      {"sector": "XLF", "return": 0.012, "vs_spy": -0.004, "rank": 3}
    ],
    "20day": [...],
    "60day": [...]
  },
  "rotation_signals": {
    "growth_vs_value": {"signal": "growth_leadership", "strength": 0.75},
    "cyclical_vs_defensive": {"signal": "cyclical_strength", "strength": 0.60},
    "risk_sentiment": "risk_on",
    "leadership_change": false
  },
  "flow_analysis": {
    "volume_leaders": ["XLK", "XLF"],
    "volume_laggards": ["XLU", "XLRE"],
    "smart_money_flow": "into_tech_financials",
    "rotation_velocity": "moderate"
  },
  "predictions": {
    "leadership_sustainability": {"XLK": 0.75, "XLY": 0.65},
    "rotation_probability": 0.35,
    "next_rotation_target": "XLF",
    "timeframe": "2-4_weeks"
  }
 },
 "alerts": [
  {
    "priority": "medium",
    "message": "Technology sector showing momentum exhaustion signals",
    "confidence": 0.68
  }
 ]
}
```

Market Regime Classification Agent

Overall Market Environment Assessment

Synthesizes data across indices, sectors, and volatility to classify the overall market regime and predict regime transitions.

Market Regime Agent Specifications:

Data Requirements:
├── Major Indices: SPY, QQQ, IWM performance and technical indicators
├── Volatility Environment: VIX levels, term structure, volatility products
├── Sector Analysis: Sector rotation patterns and breadth indicators
├── Economic Context: Interest rates, economic indicators, calendar events
└── Market Structure: Volume patterns, breadth indicators, sentiment measures

Regime Classifications:
├── Trend Regimes:
│ ├── Strong Bull: Sustained uptrend with broad participation
│ ├── Moderate Bull: Uptrend with some rotation and consolidation
│ ├── Range-Bound: Sideways movement within defined levels
│ ├── Moderate Bear: Downtrend with oversold bounces
│ └── Strong Bear: Sustained downtrend with broad selling
├── Volatility Regimes:
│ ├── Low Volatility: VIX <15, stable conditions
│ ├── Normal Volatility: VIX 15-25, average conditions
│ ├── High Volatility: VIX 25-35, stressed conditions
│ └── Crisis Volatility: VIX >35, extreme stress
└── Risk Regimes:
 ├── Risk-On: Growth sectors leading, leveraged products strong
 ├── Risk-Off: Defensive sectors leading, safe haven flows
 ├── Transition: Mixed signals, regime uncertainty
 └── Crisis: Flight to quality, correlation breakdown

Analysis Framework:
├── Technical Trend Analysis:
│ ├── Price vs Moving Averages: 20, 50, 200-day relationships
│ ├── Trend Strength: ADX readings across major indices
│ ├── Momentum Indicators: RSI, MACD convergence/divergence
│ └── Support/Resistance: Key level validation and breakouts
├── Breadth Analysis:
│ ├── Sector Participation: Percentage of sectors in uptrend
│ ├── Cross-Asset Confirmation: Bonds, commodities, currencies
│ ├── Leverage Performance: 2x/3x ETF relative strength
│ └── Defensive vs Cyclical: Relative sector performance ratios
├── Volatility Integration:
│ ├── VIX Regime Input: Volatility environment classification
│ ├── Term Structure: Normal vs inverted volatility curves
│ ├── Realized vs Implied: Historical vs forward volatility
│ └── Volatility Clustering: Persistence of vol regimes
└── Economic Context:
 ├── Interest Rate Environment: Fed policy and yield curve shape
 ├── Economic Calendar: Upcoming events and catalyst timing
 ├── Earnings Season: Corporate fundamentals and guidance trends
 └── Geopolitical Events: External shock probability assessment

Regime Transition Signals:
├── Early Warning Indicators:
│ ├── Volatility regime changes preceding trend changes

```
│   ├── Sector rotation patterns signaling regime shifts
│   ├── Breadth divergences vs price action
│   └── Cross-asset correlation breakdowns
├── Confirmation Signals:
│   ├── Multiple timeframe alignment of trend indicators
│   ├── Volume confirmation of directional moves
│   ├── Economic data supporting regime change thesis
│   └── Technical level breaks with follow-through
└── False Signal Filters:
    ├── Single-day moves vs sustained pattern changes
    ├── Low-volume breakouts vs high-conviction moves
    ├── Isolated indicator signals vs broad confirmation
    └── Historical regime persistence vs transition probability
```

Output Format:


```
{
  "agent_type": "market_regime",
  "symbol": "MARKET_OVERALL",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "confidence_score": 0.79,
  "insights": {
    "current_regime": {
      "trend_regime": "moderate_bull",
      "volatility_regime": "normal",
      "risk_regime": "risk_on",
      "overall_classification": "healthy_bull_market",
      "regime_strength": 0.75
    },
    "regime_characteristics": {
      "trend_indicators": {
        "spy_vs_ma200": 0.08,
        "adx_strength": 28.5,
        "momentum_score": 0.72
      },
      "breadth_indicators": {
        "sectors_in_uptrend": 0.78,
        "cross_asset_confirmation": 0.65,
        "leverage_performance": 1.15
      },
      "volatility_context": {
        "vix_classification": "normal",
        "term_structure": "normal_contango",
        "vol_clustering": "stable"
      }
    },
    "regime_sustainability": {
      "current_regime_probability": 0.75,
      "sustainability_timeframe": "4_to_8_weeks",
      "key_support_levels": [4450, 4380, 4320],
      "regime_threats": ["fed_policy_shift", "earnings_disappointment"]
    },
    "transition_probabilities": {
      "to_strong_bull": 0.25,
```

```
      "to_range_bound": 0.30,
      "to_moderate_bear": 0.15,
      "regime_change_timeframe": "2_to_6_weeks"
    },
    "trading_implications": {
      "strategy_environment": "bullish_momentum_favorable",
      "risk_management": "moderate_stop_losses",
      "sector_allocation": "growth_overweight",
      "volatility_strategy": "neutral_to_short_vol"
    }
  },
  "alerts": [
    {
      "priority": "low",
      "message": "Market regime stable, no transition signals detected",
      "confidence": 0.79
    }
  ]
}
```

---

8. AGENT COORDINATION PATTERNS

Simple MVP Coordination Strategy

Agents operate independently and communicate through the Storage Coordinator rather than direct inter-agent communication. This keeps the architecture simple while allowing for future enhancement.

```
Coordination Architecture:

Independent Agent Operation:
├── Each agent fetches its own data requirements
├── Agents perform analysis without inter-agent dependencies
├── All outputs flow through Storage Coordinator
├── Agents can reference other agents' stored insights when needed
└── No real-time inter-agent communication required for MVP

Data Flow Pattern:
Market Data → Agent Analysis → Storage Coordinator → Redis/Supabase
                                     ↓
                          Other Agents + Frontend

Insight Consumption:
├── Agents can query Storage Coordinator for related insights
├── VIX Agent output influences Market Regime Agent context
├── Sector Rotation feeds into Market Regime classification
├── Cross-reference through Storage Coordinator API calls
└── No blocking dependencies - agents work with available data

Timing Coordination:
├── All agents run on independent schedules
├── VIX Agent: Every 5 minutes during market hours
├── Sector Rotation Agent: Every 15 minutes during market hours
├── Market Regime Agent: Every 30 minutes (synthesis role)
├── Off-hours: Reduced frequency or pause based on agent type
└── Event-driven execution: News events, market moves, volatility spikes

Future Enhancement Path:
├── Add direct agent-to-agent communication when needed
├── Implement workflow orchestration for complex multi-agent tasks
├── Create agent hierarchy for coordinated decision-making
├── Build consensus mechanisms for conflicting agent signals
└── Develop learning loops for cross-agent optimization
```

---

9. FRONTEND DASHBOARD SPECIFICATIONS

Real-Time Market Intelligence Interface

The dashboard provides users with live market data, AI-generated insights, and alert notifications in a clean, actionable format.

Dashboard Architecture:

Core Components:
├── Market Overview Panel: Live prices and basic charts for 21 symbols
├── AI Insights Panel: Current agent analysis and confidence scores
├── Alert Center: Real-time notifications and signal changes
├── Historical Performance: Agent accuracy tracking and insight trends
└── Subscription Management: User preferences for alerts and data feeds

Market Overview Panel:
├── Symbol Grid: Live prices, % change, volume for all tracked symbols
├── Sector Performance: XL* ETF heatmap showing relative strength
├── Volatility Dashboard: VIX complex with trend indicators
├── Market Regime Indicator: Current classification with confidence
└── Quick Charts: Sparklines showing intraday patterns

AI Insights Panel:
├── VIX Analysis: Current volatility environment and predictions
├── Sector Rotation: Leadership rankings and rotation signals
├── Market Regime: Overall market classification and sustainability
├── Signal Strength: Confidence scores and supporting evidence
└── Insight Timeline: Historical agent outputs and accuracy tracking

Alert Center:
├── Priority Alerts: High-importance market regime changes
├── Symbol Alerts: Individual symbol threshold breaches
├── Agent Notifications: New insights and analysis updates
├── System Alerts: Data quality issues, connection problems
└── Alert History: Previous notifications and user actions taken

Technical Requirements:
├── Real-time Updates: Supabase subscriptions for live data
├── Performance: <2 second page load, <500ms for updates
├── Mobile Responsive: Tablet and phone optimization
├── Accessibility: WCAG 2.1 compliance for screen readers
└── Browser Support: Modern browsers (Chrome, Firefox, Safari, Edge)

Data Integration:
├── Supabase Real-time: Automatic updates for AI insights
├── Redis WebSocket: Live market data streaming
├── Storage Coordinator API: Historical data and custom queries
├── Error Handling: Graceful degradation when data unavailable
└── Caching: Intelligent local caching for performance

User Experience:
├── Customization: User-defined symbol watchlists and alert preferences
├── Filtering: Filter insights by confidence, timeframe, agent type
├── Export: Download historical insights and performance data
├── Notifications: Browser notifications for critical alerts
└── Help System: Integrated help and agent explanation tooltips

10. API SPECIFICATIONS

External API Access Layer

Provides programmatic access to market data, AI insights, and real-time alerts for external systems and integrations.

API Architecture:

RESTful Endpoints:

```
├── Authentication: JWT-based authentication via Supabase Auth
├── Rate Limiting: Tiered limits based on user plan and endpoint
├── Versioning: /api/v1/ prefix with backward compatibility
├── Documentation: OpenAPI/Swagger specification with examples
└── Error Handling: Consistent error responses with detailed messages
```

Core Endpoints:

Market Data Access:

```
├── GET /api/v1/market-data/{symbol}
│   ├── Query Parameters: timeframe, start_date, end_date, limit
│   ├── Response: Historical OHLCV data with quality scores
│   ├── Rate Limit: 100 requests/minute
│   └── Caching: 1-minute cache for recent data
├── GET /api/v1/market-data/live/{symbol}
│   ├── Response: Current quote with timestamp and quality
│   ├── WebSocket Upgrade: /ws/market-data for real-time streaming
│   ├── Rate Limit: 300 requests/minute
│   └── Fallback: Redis -> Supabase data retrieval
```

AI Insights Access:

```
├── GET /api/v1/insights/{agent_type}
│   ├── Query Parameters: symbol, confidence_min, timeframe, limit
│   ├── Response: Agent insights with metadata and performance
│   ├── Rate Limit: 60 requests/minute
│   └── Filtering: Data quality and confidence thresholds
├── GET /api/v1/insights/{agent_type}/{symbol}/latest
│   ├── Response: Most recent insight for specific agent/symbol
│   ├── Rate Limit: 120 requests/minute
│   └── Caching: 30-second cache for frequently requested insights
├── POST /api/v1/insights/query
│   ├── Request Body: Complex query with multiple filters
│   ├── Response: Paginated results with aggregation options
│   ├── Rate Limit: 30 requests/minute
│   └── Authentication: Requires valid API key
```

Alert Management:

```
├── POST /api/v1/alerts/subscribe
│   ├── Request Body: Alert criteria and delivery preferences
│   ├── Response: Subscription ID and confirmation
│   ├── Delivery Methods: Webhook, email, SMS
│   └── Rate Limit: 10 subscriptions/hour
├── GET /api/v1/alerts/history
│   ├── Query Parameters: start_date, end_date, priority, limit
│   ├── Response: Historical alerts with delivery status
│   ├── Rate Limit: 60 requests/minute
│   └── Retention: 90-day alert history
```

WebSocket Streams:

```
├── /ws/market-data: Real-time market data streaming
├── /ws/insights: Live AI insights and analysis updates
├── /ws/alerts: Real-time alert notifications
├── Authentication: JWT token in connection headers
└── Message Format: JSON with consistent schema

Response Formats:
├── Success Response (200):
{
  "status": "success",
  "data": {...},
  "metadata": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "request_id": "uuid4",
    "cache_status": "hit|miss|bypass"
  }
}
├── Error Response (4xx/5xx):
{
  "status": "error",
  "error": {
    "code": "INVALID_SYMBOL",
    "message": "Symbol XYZ not found in tracked symbols",
    "details": {...}
  },
  "metadata": {
    "timestamp": "2024-01-15T10:30:00.000Z",
    "request_id": "uuid4"
  }
}

Rate Limiting:
├── Tier Structure:
│   ├── Free Tier: 1000 requests/day, 60/minute burst
│   ├── Basic Tier: 10K requests/day, 300/minute burst
│   ├── Pro Tier: 100K requests/day, 1000/minute burst
│   └── Enterprise: Custom limits based on needs
├── Headers: X-RateLimit-Remaining, X-RateLimit-Reset
├── Enforcement: 429 status code with retry-after header
└── Monitoring: Usage tracking and overage alerts

Security:
├── Authentication: JWT tokens with expiration
├── Authorization: Role-based access control (RBAC)
├── HTTPS Only: TLS 1.3 encryption for all endpoints
├── Input Validation: Strict parameter validation and sanitization
├── CORS: Configurable cross-origin resource sharing
└── Audit Logging: Complete API access logs with user attribution
```

---

11. DEPLOYMENT & MONITORING

Production Deployment Strategy

Deployment architecture optimized for reliability, performance, and maintainability using managed services.

```
Deployment Architecture:

Service Deployment:
├── Supabase: Managed PostgreSQL with automatic scaling
├── Railway: FastAPI microservices with auto-deployment
├── Vercel: Frontend hosting with global CDN
├── Redis: Railway-managed Redis instance
└── Monitoring: Integrated monitoring across all services

Railway Deployment Configuration:
├── Service Structure:
│   ├── alpaca-data-service: Market data ingestion
│   ├── news-intelligence-service: Brave Search integration
│   ├── economic-data-service: FRED API integration
│   ├── storage-coordinator: Central data distribution
│   ├── vix-analysis-agent: VIX ecosystem analysis
│   ├── sector-rotation-agent: Sector performance analysis
│   ├── market-regime-agent: Overall market classification
│   └── api-gateway: External API access layer
├── Deployment Pipeline:
│   ├── Git Integration: Automatic deploys from main branch
│   ├── Environment Variables: Secure config management
│   ├── Health Checks: Service health monitoring
│   ├── Auto-scaling: CPU/memory-based scaling
│   └── Rollback: Instant rollback to previous versions

Environment Management:
├── Development: Local development with Docker Compose
├── Staging: Railway staging environment for testing
├── Production: Railway production with monitoring
├── Configuration: Environment-specific variables
└── Secrets: Secure API key and credential management

Monitoring & Observability:
├── Application Monitoring:
│   ├── Railway Metrics: Service performance and resource usage
│   ├── Supabase Analytics: Database performance and queries
│   ├── Vercel Analytics: Frontend performance and user metrics
│   ├── Custom Metrics: Business logic and agent performance
│   └── Error Tracking: Exception monitoring and alerting
├── Infrastructure Monitoring:
│   ├── Service Health: Endpoint monitoring and availability
│   ├── Database Performance: Query performance and connection pools
│   ├── Redis Monitoring: Memory usage, connection counts, latency
│   ├── API Performance: Response times, error rates, throughput
│   └── External Dependencies: Third-party API status and latency
├── Business Metrics:
│   ├── Agent Performance: Insight accuracy and confidence tracking
│   ├── Data Quality: Source reliability and completeness metrics
│   ├── User Engagement: Dashboard usage and feature adoption
│   ├── API Usage: External API consumption and rate limiting
│   └── Alert Effectiveness: Alert timing and user response rates
```

```
Performance Requirements:
├── Availability: 99.9% uptime target (43 minutes downtime/month)
├── Response Times: <200ms API responses, <2s dashboard loads
├── Throughput: Handle 1000+ concurrent users, 10K+ API calls/hour
├── Data Latency: <30s from market event to insight generation
├── Scalability: Auto-scale to handle 10x traffic spikes
└── Recovery: <5 minute recovery time from service failures

Alerting Strategy:
├── Critical Alerts:
│   ├── Service downtime >2 minutes
│   ├── Database connection failures
│   ├── External API failures affecting data collection
│   ├── Redis memory usage >80%
│   └── Error rates >5% over 5-minute window
├── Warning Alerts:
│   ├── Response times >500ms sustained over 2 minutes
│   ├── Data quality scores <0.8 for any source
│   ├── Agent confidence scores dropping <0.6
│   ├── Unusual traffic patterns or potential abuse
│   └── Approaching rate limits on external APIs
├── Information Alerts:
│   ├── Successful deployments and configuration changes
│   ├── Scheduled maintenance and updates
│   ├── Weekly performance and usage reports
│   └── Monthly cost and resource utilization summaries

Backup & Recovery:
├── Database Backups:
│   ├── Supabase: Automatic daily backups with point-in-time recovery
│   ├── Redis: Daily snapshots with 7-day retention
│   ├── Configuration: Version-controlled environment configs
│   └── Recovery Testing: Monthly backup restoration tests
├── Data Recovery:
│   ├── Market Data: Re-ingestion from Alpaca historical APIs
│   ├── AI Insights: Regeneration from stored market data
│   ├── User Data: Restore from Supabase backups
│   └── Configuration: Redeploy from Git repository
├── Disaster Recovery:
│   ├── Service Recreation: Automated service provisioning
│   ├── Data Restoration: Prioritized data recovery procedures
│   ├── DNS Failover: Backup service endpoints
│   ├── Communication Plan: User notification procedures
│   └── RTO/RPO: 4-hour recovery time, 1-hour data loss maximum

Security Measures:
├── Access Control:
│   ├── Multi-factor authentication for admin access
│   ├── Role-based permissions for team members
│   ├── API key rotation and secure storage
│   ├── Network security groups and firewall rules
│   └── Regular security audits and vulnerability scans
```

```
├── Data Protection:
│   ├── Encryption at rest and in transit
│   ├── PII data handling and compliance
│   ├── Secure API communication protocols
│   ├── User data privacy and GDPR compliance
│   └── Secure development practices and code reviews
```

---

12. PERFORMANCE & SCALING CONSIDERATIONS

Scalability Framework

The architecture is designed to scale horizontally as user base and data volume grow, with clear upgrade paths for each component.

```
Scaling Triggers & Responses:

User Growth Scaling:
├── 0-1K Users: Current MVP configuration
├── 1K-10K Users:
│   ├── Increase Railway service instances
│   ├── Upgrade Supabase to Pro plan
│   ├── Add Redis clustering
│   └── Implement CDN for static assets
├── 10K-50K Users:
│   ├── Implement database read replicas
│   ├── Add load balancing across service instances
│   ├── Upgrade to dedicated Redis instances
│   └── Consider microservice decomposition
├── 50K+ Users:
│   ├── Multi-region deployment
│   ├── Database sharding strategies
│   ├── Advanced caching layers
│   └── Consider migration to Kubernetes

Data Volume Scaling:
├── Current: ~21 symbols, 1-minute data, 30-day retention
├── Medium Scale: 100+ symbols, multiple timeframes
│   ├── Implement time-series database (TimescaleDB)
│   ├── Add data partitioning by symbol/date
│   ├── Optimize query patterns and indexing
│   └── Implement intelligent data archival
├── Large Scale: 1000+ symbols, tick data, options chains
│   ├── Distributed time-series storage
│   ├── Stream processing frameworks (Apache Kafka)
│   ├── Advanced data compression and storage
│   └── Edge computing for regional data processing

Performance Optimization:
├── Database Optimization:
│   ├── Query optimization and index tuning
│   ├── Connection pooling and prepared statements
│   ├── Read replica distribution
│   └── Automated performance monitoring
├── Caching Strategy:
│   ├── Multi-level caching (Redis, CDN, browser)
│   ├── Intelligent cache invalidation
│   ├── Precomputed aggregations
│   └── Cache warming strategies
├── API Optimization:
│   ├── Response compression and pagination
│   ├── GraphQL for efficient data fetching
│   ├── API gateway for rate limiting and routing
│   └── Asynchronous processing for heavy operations

Cost Optimization:
├── Resource Right-Sizing:
```

```
│   ├── Continuous monitoring of resource utilization
│   ├── Auto-scaling based on actual demand
│   ├── Reserved instance pricing for predictable workloads
│   └── Spot instance usage for batch processing
├── Data Storage Optimization:
│   ├── Automated data lifecycle management
│   ├── Compression for historical data
│   ├── Tiered storage for different access patterns
│   └── Regular cleanup of expired data
├── Service Optimization:
│   ├── Function-based pricing for variable workloads
│   ├── Edge computing to reduce bandwidth costs
│   ├── Efficient API design to minimize calls
│   └── Bulk operations for data processing

Migration Paths:
├── Database Migration:
│   ├── PostgreSQL → TimescaleDB for time-series optimization
│   ├── Single database → Microservice databases
│   ├── Supabase → Self-hosted for cost optimization
│   └── Regional database distribution
├── Infrastructure Migration:
│   ├── Railway → Kubernetes for advanced orchestration
│   ├── Monolithic services → Microservices architecture
│   ├── Single region → Multi-region deployment
│   └── Cloud provider optimization
├── Architecture Evolution:
│   ├── Synchronous → Event-driven architecture
│   ├── Request-response → Streaming data processing
│   ├── Single-tenant → Multi-tenant optimization
│   └── Batch processing → Real-time stream processing
```

CONCLUSION

This comprehensive specification provides the technical foundation for building an MVP data pipeline and AI agent architecture focused on market intelligence and signal generation. The design emphasizes simplicity for rapid development while maintaining clear paths for scaling and enhancement as the platform grows.

The architecture leverages managed services to minimize operational complexity, uses proven technologies for reliability, and implements intelligent data management for both real-time access and historical analysis. The AI agent system provides sophisticated market analysis through specialized agents that operate independently while sharing insights through a central coordination layer.

Key benefits of this approach include rapid time-to-market, scalable infrastructure, intelligent market analysis, and a solid foundation for future enhancement with trading strategies and advanced AI capabilities.